TaskVerse API Documentation

REST API reference for the TaskVerse backend (Express 5, MongoDB, Upstash Redis, Cloudinary). Backend and frontend are both deployed on Vercel.

1. Overview

Item	Value
Backend base URL	`https://<your-backend>.vercel.app`
Frontend URL	`https://<your-frontend>.vercel.app` (invitation emails currently link to `https://taskverse-frontend-ochre.vercel.app/app`)
Content type	`application/json` (multipart/form-data for uploads)
Auth	`Authorization: Bearer <accessToken>`
Rate limit	300 requests / 15 min per IP (`429` with `{ "message": "Too many requests, please try again in a few minutes" }`)
Health check	`GET /` or `GET /api` returns `{ "message": "TaskVerse API is running", "status": "healthy" }`

Route prefixes. Every router is mounted twice, so `/auth/signIn` and `/api/auth/signIn` are equivalent. Use whichever your frontend config prefers; this document shows the short form.

1.1 Response format

Success (`2xx`):

```
{ "message": "Task created successfully", "data": { } }
```

`data` is omitted for some actions (for example confirm email).

Error (`4xx` / `5xx`):

```
{ "message": "Task not found" }
```

Validation error (`400`), produced by Joi:

```
{
  "message": "Validation Error",
  "errors": [ { "key": "body", "path": "email", "message": "\"email\" must be
a valid email" } ]
}
```

When `NODE_ENV` is not `production` , error bodies also include a `stack` field.

1.2 Status codes

Code	Meaning
200 / 201	Success / created
400	Validation failure, or a generic thrown error (most auth and OTP errors land here)
401	Missing, invalid, expired or revoked token
403	Authenticated but not allowed (team and task permission checks)
404	Resource not found
409	Conflict (already exists, already a team member, already assigned)
429	Rate limited

1.3 Roles and permissions

- **System role** (`member.role`): `admin` or `member` . Public sign-up creates an `admin` . Admins create `member` accounts.
- **Team manager**: system admin, the team creator (`addedBy`), or anyone in the team's admin list (`teamAdims`).
- **Task manager**: system admin, the task creator, or a manager of the task's team.
- **Task access** (comments, subtasks, attachments): task manager, any team member, or any member assigned to the task.

1.4 ID format

All IDs are MongoDB ObjectIds: 24-character hex strings.

2. Authentication

2.1 How tokens work

- `accessToken`: JWT valid for **20 minutes**. Send it on every protected request.
- `refreshToken`: JWT valid for **7 days**. Exchange it at `POST /auth/refreshToken`.
- Logging out, changing the password, or resetting the password invalidates tokens (see each endpoint).
- Because the frontend is on a different origin, keep tokens in memory or secure storage and send them in the `Authorization` header. CORS is configured with `credentials: true`.

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

2.2 Password rules

Minimum 8 characters with at least one lowercase letter, one uppercase letter, one digit and one special character. Allowed characters are letters, digits and only these specials: `@ $ ! % * ? &`. Other symbols are rejected.

2.3 Endpoints

All auth endpoints use `POST`.

POST `/auth/signUp`

Register an admin account and email a 6-digit OTP (valid 10 minutes). Content type: `multipart/form-data` (plain JSON also works when there is no picture).

Field	Type	Rules
`fname`	string	required, 2-50 chars
`lname`	string	required, 2-50 chars
`email`	string	required, valid email (lowercased)

Field	Type	Rules
`password`	string	required, see password rules
`confirmPassword`	string	required, must equal `password`
`gender`	string	required, `male` or `female`
`profilePicture`	file	optional, PNG/JPEG, max 5 MB

Response `201`:

```
{ "message": "Admin user created successfully", "data": { "email": "jane@example.com" } }
```

Errors: `400` `User already exists`, validation errors.

POST `/auth/confirmSignUp`

Confirm the email with the OTP.

```
{ "email": "jane@example.com", "otp": "123456" }
```

Response `200`: `{ "message": "Email confirmed successfully" }`. Errors (`400`): `User not found`, `Otp not found`, `Invalid Otp`.

POST `/auth/resendOtp`

Body: `{ "email": "jane@example.com" }`. Sends a new OTP (valid 15 minutes) to an unconfirmed account. After 3 resends the email is blocked for 10 minutes. Response `200`: `Otp sent successfully`. Errors (`400`): `User not found`, `You have exceeded the maximum number of attempts ...`.

POST `/auth/signIn`

```
{ "email": "jane@example.com", "password": "Passw0rd!" }
```

Response `200`:

```
{
  "message": "login successfully",
  "data": { "accessToken": "<jwt>", "refreshToken": "<jwt>" }
}
```

Errors (`400`): `User not found` (also returned for unconfirmed accounts), `Invalid password` .

POST `/auth/sign&signin/gmail`

Google sign-in. Body: `{ "idToken": "<Google ID token>" }` . Creates the account on first use (already confirmed) and returns `{ accessToken, refreshToken }` like `signIn` . Errors (`400`): `Google ID token is required` , `Google email is not verified` , `Email already exists. Please sign in using your password.` See the known issue in section 8 about Google accounts and protected routes.

POST `/auth/profile` (auth)

Returns the signed-in member without the password. Note this is a `POST` , not a `GET` .

POST `/auth/refreshToken` (auth)

Send the **refresh token** as the Bearer token. Returns a new pair and revokes the one you sent.

```
{ "message": "Tokens refreshed successfully", "data": { "accessToken": "
<jwt>", "refreshToken": "<jwt>" } }
```

POST `/auth/logout` (auth)

- No query: revokes the current token only.
- `?flag=all` : signs the member out everywhere (all tokens issued before now stop working).

Response `200` : `Logout successfully` .

POST `/auth/forgotPassword`

Body: `{ "email": "..." }` . Emails a reset OTP valid for 15 minutes. Errors (`400`): `User not found` .

POST `/auth/resetPassword`

```
{ "email": "jane@example.com", "otp": "123456", "password": "NewPassw0rd!",
"confirmPassword": "NewPassw0rd!" }
```

Response `200` : `Password changed successfully` . All existing sessions are invalidated. Errors (`400`): `Otp not found` , `Invalid otp` , `User not found` .

POST `/auth/changePassword` (auth)

```
{ "oldPassword": "Passw0rd!", "newPassword": "NewPassw0rd!",
"confirmPassword": "NewPassw0rd!" }
```

Response `200`: `Password changed successfully`. Other sessions are invalidated, so sign in again. Errors (`400`): `Invalid old password`.

3. Members

Method	Path	Access	Description
GET	`/member` or `/member/all`	any signed-in user	All members (no passwords), newest first
GET	`/member/profile`	any signed-in user	The caller's own profile
POST	`/member/createMember`	system admin	Create a `member` account

POST `/member/createMember`

`multipart/form-data`. The new account is created as role `member`, already confirmed, and an invitation email containing the sign-in details is sent to it.

Field	Type	Rules
`fname`, `lname`	string	required
`email`	string	required, valid email, must be unused
`password`, `confirmPassword`	string	required, must match
`gender`	string	required, `male` or `female`
`profilePicture`	file	optional, PNG/JPEG, max 5 MB

Response `201`: `Member created successfully` with the created member in `data`. Errors (`400`): `Member already exists`, `Password and confirm password do not match`, `unauthorized` (caller is not an admin).

Member object (returned by list and profile endpoints):

```
{
  "_id": "665f...",
  "fname": "Jane",
  "lname": "Doe",
  "email": "jane@example.com",
  "gender": "female",
  "role": "admin",
  "provider": "system",
  "confirmed": true,
  "profilePicture": "https://res.cloudinary.com/...",
  "teams": [ { "teamId": "665f...", "role": "teamAdmin" } ],
  "addedBy": null,
  "createdAt": "2026-11-01T10:00:00.000Z",
  "updatedAt": "2026-11-01T10:00:00.000Z"
}
```


4. Teams

The team field holding admins is spelled `teamAdims` in the database and responses. The API keeps this spelling.

Method	Path	Access	Description
GET	`/team` or `/team/all`	signed-in	System admins see all teams. Others see teams they belong to, created, or administer
POST	`/team/createTeam`	system admin	Create a team
POST	`/team/addMemberToTeam`	team manager	Add one or many members
POST	`/team/removeMemberFromTeam`	team manager	Remove one or many members
POST	`/team/updateMemberRoleInTeam`	team manager	Promote or demote a member
POST	`/team/updateTeamName`	team manager	Rename a team

Method	Path	Access	Description
POST	`/team/deleteTeam`	team manager	Delete a team

`GET /team` returns teams with `teamMembers` and `addedBy` populated with `fname lname email`.

Team object:

```
{
  "_id": "665f...",
  "name": "Design",
  "addedBy": "665f...",
  "teamAdims": ["665f..."],
  "teamMembers": ["665f..."],
  "createdAt": "...",
  "updatedAt": "..."
}
```

POST `/team/createTeam`

Body: `{ "name": "Design" }` (2-50 chars). The creator becomes team admin and member. Response `201`: `Team created successfully` with the team.

POST `/team/addMemberToTeam`

Field	Type	Rules
`teamId`	string	required
`memberId` / `memberIds` / `member_email`	string / string[] / string	at least one is required
`role`	string	`member` (default), `admin`, `teamMember`, `teamAdmin`. The `admin` values also add the person to `teamAdims`

- Single (`memberId` or `member_email`): `200` with `data: { teamId, memberId }`. Errors: `404` `Team not found` / `Member not found`, `403` `You are not the team admin`, `409` `Member already in team`.

- Bulk (`memberIds`): `200` with `data: { teamId, memberIds: [added...] }`. Invalid IDs, unknown members and existing members are skipped silently.

POST `/team/removeMemberFromTeam`

Body: `{ "teamId": "...", "memberId": "..." }` or `{ "teamId": "...", "memberIds": ["..."] }`. Removes the member from the team's members and admins. Errors: `404` `Team not found` / `Member not found` / `Member not found in team`, `403`.

POST `/team/updateMemberRoleInTeam`

Body: `{ "teamId": "...", "memberId": "...", "role": "admin" }`. Accepts the same role values as add. Response `200`: `data: { teamId, memberId, role }`.

POST `/team/updateTeamName`

Body: `{ "teamId": "...", "name": "New name" }`. Response `200` with the updated team.

POST `/team/deleteTeam`

Body: `{ "teamId": "..." }`. Response `200`: `data: { teamId }`. Team references are removed from members, and the team of its tasks is set to `null` (tasks are kept).